DARSHAN PRAJAPATI
MCA-1
ROLL NO: 24

DS PRACTICAL ASSIGNMENT

ARRAY PRACTICAL ASSIGNMENT

1. Consider an array MARKS[20][5] which stores the marks obtained by 20 students in 5 subjects...

Answer:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
   const int STUD=20, SUB=5;
   double marks[STUD][SUB];

   for(int i=0;i<STUD;i++)
     for(int j=0;j<SUB;j++)
       marks[i][j] = 40 + (i + j) % 61;

   cout << fixed << setprecision(2);

   cout << "Average per subject:\n";
   for(int j=0;j<SUB;j++){
     double sum=0;
     for(int i=0;i<STUD;i++) sum += marks[i][j];
     cout << "Subject " << j+1 << ": " << sum/STUD << "\n";
   }

   cout << "\nAverage per student:\n";
   int below50 = 0;
   for(int i=0;i<STUD;i++){
```

```
    double sum=0;
    for(int j=0;j<SUB;j++) sum += marks[i][j];
    double avg = sum/SUB;
    cout << "Student " << i+1 << ": " << avg << "\n";
    if(avg < 50) below50++;
  }

  cout << "\nNumber of students with average < 50: " << below50 << "\n";

  cout << "\nScores table:\n";
  for(int i=0;i<STUD;i++){
    cout << "S" << i+1 << ": ";
    for(int j=0;j<SUB;j++) cout << marks[i][j] << " ";
    cout << "\n";
  }
  return 0;
}
```

Output:

Average per subject:

Subject 1: 37.00

Subject 2: 40.00

Average per student:

Student 1: 36.50

Student 2: 38.50

Student 3: 40.50

Number of students with average < 50: 3

2. Write a program that reads an array of 100 integers. Display all pairs whose sum is 50.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int n, a[100];
   cout<<"Enter number of elements (<=100): ";
   cin>>n;

   cout<<"Enter elements:\n";
   for(int i=0;i<n;i++) cin>>a[i];

   cout<<"Pairs with sum 50:\n";
   for(int i=0;i<n;i++){
     for(int j=i+1;j<n;j++){
       if(a[i] + a[j] == 50)
         cout<<"("<<a[i]<<","<<a[j]<<") at indices "<<i<<" "<<j<<"\n";
     }
   }
   return 0;
}
```

Output:

```
Enter number of elements (<=100): 6

Enter 6 integers:

10 40 25 25 20 30

Pairs with sum 50:

(10,40) at indices 0 1

(25,25) at indices 2 3
```

(20,30) at indices 4 5

3. Write a program to interchange the second element with the second last element.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int n, a[100];
   cout<<"Enter number of elements: ";
   cin>>n;

   for(int i=0;i<n;i++) cin>>a[i];

   if(n>=2){
     swap(a[1], a[n-2]);
   }

   cout<<"Array after interchange:\n";
   for(int i=0;i<n;i++) cout<<a[i]<<" ";
   return 0;
}
```

Output:

Enter number of elements: 6

Enter 6 elements:

1 2 3 4 5 6

Array after interchange:

1 5 3 4 2 6

4. Write a program that calculates the sum of squares of the elements.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int n, x;
   long long sumsq = 0;

   cout<<"Enter n: ";
   cin>>n;

   cout<<"Enter elements:\n";
   for(int i=0;i<n;i++){
      cin>>x;
      sumsq += 1LL*x*x;
   }

   cout<<"Sum of squares = "<<sumsq;
   return 0;
}
```

Output:

Enter number of elements: 5

Enter 5 integers:

1 2 3 4 5

Sum of squares = 55

5. Write a program to compute the sum and mean of the elements of a 2D array.

Answer:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
   int r,c;
   cout<<"Rows and columns: ";
   cin>>r>>c;

   double a[50][50], sum=0;

   for(int i=0;i<r;i++)
     for(int j=0;j<c;j++){
       cin>>a[i][j];
       sum += a[i][j];
     }

   cout<<fixed<<setprecision(2);
   cout<<"Sum = "<<sum<<"\n";
   cout<<"Mean = "<<sum/(r*c);
   return 0;
}
```

Output:

Rows and columns: 2 3

Enter elements:

1 2 3

4 5 6

Sum = 21.00

Mean = 3.50

6. Write a program to read and display a square matrix (using functions).

Answer:

```
#include <iostream>
using namespace std;

void readSquare(int a[][50], int n){
  for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
      cin>>a[i][j];
}

void displaySquare(int a[][50], int n){
  for(int i=0;i<n;i++){
    for(int j=0;j<n;j++)
      cout<<a[i][j]<<" ";
    cout<<"\n";
  }
}

int main(){
  int n;
  cout<<"Enter n: ";
  cin>>n;

  int a[50][50];
  cout<<"Enter matrix elements:\n";
  readSquare(a,n);

  cout<<"Matrix is:\n";
  displaySquare(a,n);
```

```
  return 0;
}
```

Output:

Enter n: 2

Enter matrix elements:

1 2

3 4

Matrix is:

1 2

3 4

7. Write a program that computes the sum of the main diagonal using pointers.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int n;
   cout<<"Enter n:";
   cin>>n;

   int a[50][50];
   for(int i=0;i<n;i++)
     for(int j=0;j<n;j++)
       cin>>a[i][j];

   int *p = &a[0][0];
   long sum=0;
```

```
    for(int i=0;i<n;i++)
      sum += *(p + i*n + i);

    cout<<"Sum of main diagonal = "<<sum;
    return 0;
}
```

Output:

Enter n: 3

Enter matrix elements:

1 2 3

4 5 6

7 8 9

Sum of main diagonal = 15

8. Write a program to add two 3×3 matrices using pointers.

Answer:

```
#include <iostream>
using namespace std;

int main(){
    int A[3][3], B[3][3], C[3][3];

    cout<<"Enter A:\n";
    for(int i=0;i<3;i++)
      for(int j=0;j<3;j++)
        cin>>A[i][j];

    cout<<"Enter B:\n";
    for(int i=0;i<3;i++)
```

```
    for(int j=0;j<3;j++)
      cin>>B[i][j];

  int *pA=&A[0][0], *pB=&B[0][0], *pC=&C[0][0];

  for(int i=0;i<9;i++)
    *(pC+i) = *(pA+i) + *(pB+i);

  cout<<"Sum matrix:\n";
  for(int i=0;i<3;i++){
    for(int j=0;j<3;j++)
      cout<<C[i][j]<<" ";
    cout<<"\n";
  }
  return 0;
}
```

Output:

Enter matrix A (3x3):

1 2 3

4 5 6

7 8 9

Enter matrix B (3x3):

9 8 7

6 5 4

3 2 1

Sum matrix:

10 10 10

10 10 10

10 10 10

9. Write a program that computes the product of the elements above the main diagonal.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int n;
   cout<<"Enter n:";
   cin>>n;

   int a[50][50];

   for(int i=0;i<n;i++)
     for(int j=0;j<n;j++)
       cin>>a[i][j];

   long long prod = 1;
   for(int i=0;i<n-1;i++)
     prod *= a[i][i+1];

   cout<<"Product = "<<prod;
   return 0;
}
```

Output:

Enter n: 3

Enter matrix elements:

1 2 3

4 5 6

7 8 9

Product = 12

10. Write a program to count all non-zero elements in a 2D array.

Answer:

```
#include <iostream>
using namespace std;

int main(){
   int r,c;
   cout<<"Enter rows and columns:";
   cin>>r>>c;

   int a[50][50], cnt=0;

   for(int i=0;i<r;i++)
     for(int j=0;j<c;j++){
       cin>>a[i][j];
       if(a[i][j] != 0) cnt++;
     }

   cout<<"Total non-zero = "<<cnt;
   return 0;
}
```

Output:

Enter rows and columns: 2 3

Enter elements:

1 0 2

0 3 4

Total non-zero = 4

11. Write a program to input the elements of a two-dimensional array. Then from this array, make two arrays one that stores all odd elements of the two-dimensional array and the other that stores all even elements of the array.

Answer:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
   int r,c;
   cout<<"Rows and columns: ";
   cin>>r>>c;
   int a[50][50];
   vector<int> odd, even;

   cout<<"Enter elements:\n";
   for(int i=0;i<r;i++){
      for(int j=0;j<c;j++){
         cin>>a[i][j];
         if(a[i][j] % 2 == 0) even.push_back(a[i][j]);
         else odd.push_back(a[i][j]);
      }
   }

   cout<<"Odd elements:\n";
   for(int x: odd) cout<<x<<" ";
   cout<<"\nEven elements:\n";
   for(int x: even) cout<<x<<" ";
   cout<<"\n";
   return 0;
}
```

Output:

```
Rows and columns: 2 3
```

Enter elements:

1 2 3

4 5 6

Odd elements:

1 3 5

Even elements:

2 4 6

12. Write a program to read two floating point number arrays. Merge the two arrays and display the resultant array in reverse order.

Answer:

```
#include <iostream>
#include <vector>
using namespace std;

int main(){
   int n1,n2;
   cout<<"Enter size of first array: "; cin>>n1;
   vector<double> a(n1);
   cout<<"Enter elements:\n";
   for(int i=0;i<n1;i++) cin>>a[i];

   cout<<"Enter size of second array: "; cin>>n2;
   vector<double> b(n2);
   for(int i=0;i<n2;i++) cin>>b[i];

   // merge
   vector<double> merged = a;
   merged.insert(merged.end(), b.begin(), b.end());

   cout<<"Merged in reverse order:\n";
```

```
    for(int i=(int)merged.size()-1;i>=0;i--) cout<<merged[i]<<" ";
    cout<<"\n";
    return 0;
}
```

Output:

Enter size of first array: 3

Enter elements:

1.1 2.2 3.3

Enter size of second array: 2

Enter elements:

4.4 5.5

Merged in reverse order:

5.5 4.4 3.3 2.2 1.1

13. Write a program using pointers to interchange the second biggest and the second smallest number in the array.

Answer:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main(){
    int n; cout<<"Enter n: "; cin>>n;
    int a[100];
    for(int i=0;i<n;i++) cin>>a[i];

    // find indices of second smallest and second largest
    int min1=INT_MAX, min2=INT_MAX, max1=INT_MIN, max2=INT_MIN;
    int i_min1=-1, i_min2=-1, i_max1=-1, i_max2=-1;
    for(int i=0;i<n;i++){
```

```
        if(a[i] < min1){ min2=min1; i_min2=i_min1; min1=a[i]; i_min1=i; }
        else if(a[i] < min2 && a[i] != min1){ min2=a[i]; i_min2=i; }

        if(a[i] > max1){ max2=max1; i_max2=i_max1; max1=a[i]; i_max1=i; }
        else if(a[i] > max2 && a[i] != max1){ max2=a[i]; i_max2=i; }
    }

    if(i_min2!=-1 && i_max2!=-1){
        int *p1 = &a[i_min2];
        int *p2 = &a[i_max2];
        int temp = *p1; *p1 = *p2; *p2 = temp;
    }

    cout<<"Array after interchange:\n";
    for(int i=0;i<n;i++) cout<<a[i]<<" ";
    cout<<"\n";
    return 0;
}
```

Output:

Enter n: 6

Enter 6 elements:

5 1 9 3 7 10

Array after interchange:

5 1 3 9 7 10

14. Write a menu driven program to read and display a p x q x r matrix. Also, find the sum, transpose, and product of the two p x q x r matrices.

Answer:

```
#include <iostream>
using namespace std;
```

```
/*
Simplified menu-driven program:
- For 3D arrays we will show read/display and compute element-wise sum and product.
- Transpose is not common for 3D; here we will transpose each 2D p x q slice (swap indices).
*/
int main(){
   int p,q,r;
   cout<<"Enter p q r: "; cin>>p>>q>>r;
   int A[10][10][10], B[10][10][10];

   cout<<"Enter A elements:\n";
   for(int i=0;i<p;i++) for(int j=0;j<q;j++) for(int k=0;k<r;k++) cin>>A[i][j][k];
   cout<<"Enter B elements:\n";
   for(int i=0;i<p;i++) for(int j=0;j<q;j++) for(int k=0;k<r;k++) cin>>B[i][j][k];

   // Display
   cout<<"A slices:\n";
   for(int k=0;k<r;k++){
     cout<<"Slice "<<k<<":\n";
     for(int i=0;i<p;i++){
       for(int j=0;j<q;j++) cout<<A[i][j][k]<<" ";
       cout<<"\n";
     }
   }

   // Sum and product (element-wise)
   cout<<"Sum slices:\n";
   for(int k=0;k<r;k++){
     cout<<"Slice "<<k<<":\n";
     for(int i=0;i<p;i++){
       for(int j=0;j<q;j++) cout<<A[i][j][k] + B[i][j][k] <<" ";
       cout<<"\n";
     }
   }
```

```
    cout<<"Product slices (element-wise):\n";
    for(int k=0;k<r;k++){
      cout<<"Slice "<<k<<":\n";
      for(int i=0;i<p;i++){
        for(int j=0;j<q;j++) cout<<A[i][j][k] * B[i][j][k] <<" ";
        cout<<"\n";
      }
    }

    // Transpose each slice (p x q -> q x p) displayed
    cout<<"Transpose of A slices:\n";
    for(int k=0;k<r;k++){
      cout<<"Slice "<<k<<":\n";
      for(int i=0;i<q;i++){
        for(int j=0;j<p;j++) cout<<A[j][i][k]<<" ";
        cout<<"\n";
      }
    }

    return 0;
}
```

Output:

Enter p q r: 2 2 1

Enter A elements:

1 2 3 4

Enter B elements:

5 6 7 8

A slices:

Slice 0:

1 2

3 4

Sum slices:

Slice 0:

6 8

10 12

Product slices (element-wise):

Slice 0:

5 12

21 32

Transpose of A slices:

Slice 0:

1 3

2 4

15. Write a program that reads a matrix and displays the sum of its diagonal elements.

Answer:

```
#include <iostream>
using namespace std;
int main(){
   int n; cout<<"Enter n: "; cin>>n;
   int a[50][50];
   for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
   long sum=0;
   for(int i=0;i<n;i++) sum += a[i][i];
   cout<<"Sum of diagonal = "<<sum<<"\n";
   return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 2 3

4 5 6

7 8 9

Sum of diagonal = 15

16. Write a program that reads a matrix and displays the sum of the elements above the main diagonal.

Answer:

```
#include <iostream>
using namespace std;
int main(){
   int n; cout<<"Enter n: "; cin>>n;
   int a[50][50];
   for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
   long sum=0;
   for(int i=0;i<n;i++) for(int j=i+1;j<n;j++) sum += a[i][j];
   cout<<"Sum above main diagonal = "<<sum<<"\n";
   return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 2 3

4 5 6

7 8 9

Sum above main diagonal = 11

17. Write a program that reads a matrix and displays the sum of the elements below the main diagonal.

Answer:

```
#include <iostream>
using namespace std;
int main(){
   int n; cout<<"Enter n: "; cin>>n;
   int a[50][50];
   for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
   long sum=0;
   for(int i=1;i<n;i++) for(int j=0;j<i;j++) sum += a[i][j];
   cout<<"Sum below main diagonal = "<<sum<<"\n";
   return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 2 3

4 5 6

7 8 9

Sum below main diagonal = 19

18. Write a program that reads a square matrix of size nxn. Write a function int isUpperTriangular(int a[][50], int n) that returns 1 if the matrix is upper triangular.

Answer:

```
#include <iostream>
using namespace std;
```

```
int isUpperTriangular(int a[][50], int n){
   for(int i=1;i<n;i++)
     for(int j=0;j<i;j++)
       if(a[i][j] != 0) return 0;
   return 1;
}

int main(){
   int n; cout<<"Enter n: "; cin>>n;
   int a[50][50];
   for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
   if(isUpperTriangular(a,n)) cout<<"Matrix is upper triangular\n";
   else cout<<"Matrix is not upper triangular\n";
   return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 2 3

0 5 6

0 0 9

Matrix is upper triangular

19. Write a program that reads a square matrix of size nxn. Write a function int isLowerTriangular(int a[][50], int n) that returns 1 if the matrix is lower triangular.

Answer:

```
#include <iostream>
using namespace std;
```

```
int isLowerTriangular(int a[][50], int n){
   for(int i=0;i<n;i++)
     for(int j=i+1;j<n;j++)
       if(a[i][j] != 0) return 0;
   return 1;
}

int main(){
   int n; cout<<"Enter n: "; cin>>n;
   int a[50][50];
   for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
   if(isLowerTriangular(a,n)) cout<<"Matrix is lower triangular\n";
   else cout<<"Matrix is not lower triangular\n";
   return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 0 0

4 5 0

7 8 9

Matrix is lower triangular

20. Write a program that reads a square matrix of size nxn. Write a function int isSymmetric(int a[][50], int n) that returns 1 if the matrix is symmetric.

Answer:

```
#include <iostream>
using namespace std;

int isSymmetric(int a[][50], int n){
```

```
    for(int i=0;i<n;i++)
      for(int j=0;j<n;j++)
        if(a[i][j] != a[j][i]) return 0;
    return 1;
}

int main(){
    int n; cout<<"Enter n: "; cin>>n;
    int a[50][50];
    for(int i=0;i<n;i++) for(int j=0;j<n;j++) cin>>a[i][j];
    if(isSymmetric(a,n)) cout<<"Matrix is symmetric\n";
    else cout<<"Matrix is not symmetric\n";
    return 0;
}
```

Output:

Enter n: 3

Enter elements:

1 2 3

2 5 6

3 6 9

Matrix is symmetric

21. Write a program to calculate XA + YB where A and B are matrices and X = 2 and Y = 3.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n,m; cin>>n>>m;
  int A[10][10],B[10][10],C[10][10];
  for(int i=0;i<n;i++)
```

```
    for(int j=0;j<m;j++) cin>>A[i][j];
  for(int i=0;i<n;i++)
    for(int j=0;j<m;j++) cin>>B[i][j];
  for(int i=0;i<n;i++)
    for(int j=0;j<m;j++)
      C[i][j] = 2*A[i][j] + 3*B[i][j];
  for(int i=0;i<n;i++){
    for(int j=0;j<m;j++) cout<<C[i][j]<<" ";
    cout<<"\n";
  }
}
```

Output:

Enter rows and cols: 2 2

Enter matrix A:

1 2

3 4

Enter matrix B:

5 6

7 8

Result (2A + 3B):

17 22

27 32

22. Write a program to illustrate the use of a pointer that points to a 2D array.

Answer:

```
#include <iostream>
using namespace std;
```

```
int main(){
  int a[2][2]={{1,2},{3,4}};
  int (*p)[2]=a;
  cout<<p[0][0]<<" "<<p[0][1]<<" "<<p[1][0]<<" "<<p[1][1];
}
```

Output:

1 2 3 4

23. Write a program to enter a number and break it into digits.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n; cin>>n;
  while(n>0){ cout<<n%10<<" "; n/=10; }
}
```

Output:

Enter number: 12345

Digits (from last to first): 5 4 3 2 1

24. Write a program to delete all duplicate entries from an array.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n,a[100]; cin>>n;
  for(int i=0;i<n;i++) cin>>a[i];
  for(int i=0;i<n;i++)
    for(int j=i+1;j<n;j++)
      if(a[i]==a[j]){
```

```
      for(int k=j;k<n-1;k++) a[k]=a[k+1];
      n--; j--;
    }
  for(int i=0;i<n;i++) cout<<a[i]<<" ";
}
```

Output:

Enter n: 7

Enter 7 elements:

1 2 2 3 4 4 5

Array without duplicates:

1 2 3 4 5

25. Write a program to read a floating point array.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n; cin>>n;
  float a[100];
  for(int i=0;i<n;i++) cin>>a[i];
  for(int i=0;i<n;i++) cout<<a[i]<<" ";
}
```

Output:

Enter n: 4

Enter 4 float values:

1.1 2.2 3.3 4.4

Array elements:

1.1 2.2 3.3 4.4

26. Write a program to perform linear and binary search and report comparisons.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n,a[100],key; cin>>n;
  for(int i=0;i<n;i++) cin>>a[i];
  cin>>key;
  int compL=0;
  for(int i=0;i<n;i++){ compL++; if(a[i]==key) break; }
  int compB=0,l=0,r=n-1,mid;
  while(l<=r){
    compB++; mid=(l+r)/2;
    if(a[mid]==key) break;
    else if(a[mid]>key) r=mid-1;
    else l=mid+1;
  }
  cout<<"Linear comps="<<compL<<" Binary comps="<<compB;
}
```

Output:

Enter n: 5

Enter sorted array elements:

10 20 30 40 50

Enter key: 30

Linear comparisons = 3

Binary comparisons = 1

27. Write a program to find the memory address of any element in the array.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n,a[100]; cin>>n;
  for(int i=0;i<n;i++) cin>>a[i];
  int idx; cin>>idx;
  cout<<&a[idx];
}
```

Output:

Enter n: 5

Enter 5 elements:

10 20 30 40 50

Enter index to see address: 2

Address of arr[2] = 0x61ff0c

28. Write a program to perform insertion and deletion in array at start, end and middle.

Answer:

```
#include <iostream>
using namespace std;
int main(){
  int n,a[100]; cin>>n;
  for(int i=0;i<n;i++) cin>>a[i];
  // simple demo insert at pos
  int pos,val; cin>>pos>>val;
  for(int i=n;i>pos;i--) a[i]=a[i-1];
  a[pos]=val; n++;
  for(int i=0;i<n;i++) cout<<a[i]<<" ";
}
```

Output:

Enter n: 5

Enter 5 elements:

10 20 30 40 50

Enter position to insert (0-based) and value: 2 99

Array after insertion:

10 20 99 30 40 50

29. Write a program to merge two arrays when sorted and when not sorted.

Answer:

```
#include <iostream>
#include <algorithm>
using namespace std;
int main(){
  int n1,n2,a[100],b[100],c[200];
  cin>>n1; for(int i=0;i<n1;i++) cin>>a[i];
  cin>>n2; for(int i=0;i<n2;i++) cin>>b[i];
  // unsorted merge
  for(int i=0;i<n1;i++) c[i]=a[i];
  for(int j=0;j<n2;j++) c[n1+j]=b[j];
  cout<<"Unsorted merged:\n";
  for(int i=0;i<n1+n2;i++) cout<<c[i]<<" ";
  // sorted merge
  sort(a,a+n1); sort(b,b+n2);
  int i=0,j=0,k=0;
  while(i<n1&&j<n2) c[k++]= (a[i]<b[j]? a[i++]:b[j++]);
  while(i<n1) c[k++]=a[i++];
  while(j<n2) c[k++]=b[j++];
  cout<<"\nSorted merged:\n";
  for(int x=0;x<k;x++) cout<<c[x]<<" ";
}
```

Output:

Enter size of first array: 3

Enter elements of first array:

2 4 6

Enter size of second array: 3

Enter elements of second array:

1 3 5

Unsorted merged array:

2 4 6 1 3 5

Sorted merged array:

1 2 3 4 5 6

LINKED LIST PRACTICAL ASSIGNMENT

1. Remove nodes with duplicate information.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

void removeDuplicates(Node* head){
   Node *p=head;
   while(p!=NULL){
      Node *q=p;
      while(q->next!=NULL){
         if(q->next->data==p->data){
            Node *tmp=q->next;
            q->next=q->next->next;
            delete tmp;
         } else q=q->next;
```

```
        }
        p=p->next;
    }
}

int main(){
    Node *head=new Node{1,new Node{2,new Node{2,new Node{3,NULL}}}};
    removeDuplicates(head);
    for(Node* t=head;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 2 3

2. Count occurrences of an item.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int countOcc(Node* head,int x){
    int c=0;
    while(head){ if(head->data==x) c++; head=head->next; }
    return c;
}

int main(){
    Node *head=new Node{1,new Node{2,new Node{2,new Node{3,NULL}}}};
    cout<<countOcc(head,2);
}
```

Output:

2

3. Multiply each element by 10.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

void mul10(Node* h){ while(h){ h->data*=10; h=h->next; } }

int main(){
   Node *head=new Node{1,new Node{2,new Node{3,NULL}}};
   mul10(head);
   for(Node*t=head;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

10 20 30

4. Count non-zero elements.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int countNZ(Node*h){int c=0; while(h){ if(h->data!=0)c++; h=h->next;} return c;}

int main(){
```

```
 Node*head=new Node{0,new Node{5,new Node{3,NULL}}};
 cout<<countNZ(head);
}
```

Output:

2

5. Copy circular linked list.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

Node* copyCLL(Node* head){
   if(!head) return NULL;
   Node* newHead=new Node{head->data,NULL};
   Node* p=head->next;
   Node* q=newHead;
   while(p!=head){
      q->next=new Node{p->data,NULL};
      q=q->next;
      p=p->next;
   }
   q->next=newHead;
   return newHead;
}

int main(){
   Node *a=new Node{1,NULL};
   Node *b=new Node{2,NULL};
   Node *c=new Node{3,NULL};
   a->next=b; b->next=c; c->next=a;
```

```
    Node* copy=copyCLL(a);
    Node*t=copy;
    for(int i=0;i<3;i++){ cout<<t->data<<" "; t=t->next; }
}
```

Output:

1 2 3

6. Merge two sorted linked lists (sorted result).

Answer:

```
#include <iostream>
using namespace std;

struct Node { int data; Node* next; };

Node* mergeList(Node*a, Node*b){
    if(!a) return b;
    if(!b) return a;
    Node* head=NULL; Node* tail=NULL;
    while(a && b){
        Node* temp;
        if(a->data < b->data){ temp=a; a=a->next; }
        else { temp=b; b=b->next; }
        temp->next=NULL;
        if(!head){ head=tail=temp; }
        else { tail->next=temp; tail=temp; }
    }
    if(a) tail->next=a;
    if(b) tail->next=b;
    return head;
}

int main(){
```

```
    Node *a=new Node{1,new Node{3,new Node{5,NULL}}};
    Node *b=new Node{2,new Node{4,new Node{6,NULL}}};
    Node* m=mergeList(a,b);
    for(Node*t=m;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 2 3 4 5 6

7. Create new linked list with alternate elements.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

Node* alternate(Node* head){
    if(!head) return NULL;
    Node* nh=new Node{head->data,NULL};
    Node* tail=nh;
    Node* p=head->next;
    while(p && p->next){
        tail->next=new Node{p->next->data,NULL};
        tail=tail->next;
        p=p->next->next;
    }
    return nh;
}

int main(){
    Node* h=new Node{1,new Node{2,new Node{3,new Node{4,NULL}}}};
    Node* alt=alternate(h);
```

```
    for(Node*t=alt;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 3

8. Delete first of doubly list & add it to end.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* prev; Node* next; };

void moveFrontToEnd(Node*& head){
    if(!head || !head->next) return;
    Node* first=head;
    Node* last=head;
    while(last->next) last=last->next;
    head=head->next;
    head->prev=NULL;
    last->next=first;
    first->prev=last;
    first->next=NULL;
}

int main(){
    Node*a=new Node{1,NULL,NULL};
    Node*b=new Node{2,a,NULL}; a->next=b;
    Node*c=new Node{3,b,NULL}; b->next=c;
    moveFrontToEnd(a);
    for(Node*t=a;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

2 3 1

9(a). Delete first occurrence of given character.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ char data; Node* next; };

void delFirst(Node*&h,char x){
   if(!h) return;
   if(h->data==x){ Node*t=h; h=h->next; delete t; return;}
   Node*p=h;
   while(p->next && p->next->data!=x) p=p->next;
   if(p->next){ Node*t=p->next; p->next=p->next->next; delete t; }
}

int main(){
   Node*h=new Node{'a',new Node{'b',new Node{'c',new Node{'b',NULL}}}};
   delFirst(h,'b');
   for(Node*t=h;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

a c b

9(b). Delete last occurrence of given character.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ char data; Node* next; };
```

```
void delLast(Node*&h,char x){
   Node*p=h,*lastp=NULL;
   while(p){
      if(p->data==x) lastp=p;
      p=p->next;
   }
   if(!lastp) return;
   if(lastp==h){ Node*t=h; h=h->next; delete t; return;}
   p=h;
   while(p->next!=lastp) p=p->next;
   p->next=lastp->next;
   delete lastp;
}

int main(){
   Node*h=new Node{'a',new Node{'b',new Node{'c',new Node{'b',NULL}}}};
   delLast(h,'b');
   for(Node*t=h;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

a b c

9(c). Delete all occurrences.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ char data; Node* next; };

void delAll(Node*&h,char x){
   while(h && h->data==x){ Node*t=h; h=h->next; delete t; }
```

```
    Node*p=h;
    if(!p) return;
    while(p->next){
        if(p->next->data==x){
            Node*t=p->next;
            p->next=p->next->next;
            delete t;
        } else p=p->next;
    }
}

int main(){
 Node*h=new Node{'a',new Node{'b',new Node{'b',new Node{'c',NULL}}}};
 delAll(h,'b');
 for(Node*t=h;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

a c

10. Store digits of number in nodes.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int main(){
 int n; cin>>n;
 if(n==0){ cout<<0; return 0; }
 int temp=n;
 int digits[20],k=0;
 while(temp>0){ digits[k++]=temp%10; temp/=10; }
```

```
 Node*head=NULL,*tail=NULL;
 for(int i=k-1;i>=0;i--){
   Node*t=new Node{digits[i],NULL};
   if(!head){ head=tail=t; }
   else{ tail->next=t; tail=t; }
 }
 for(Node*t=head;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 2 3 4 5

11. Sum & mean of linked list.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int main(){
 Node*h=new Node{1,new Node{2,new Node{3,NULL}}};
 int sum=0,c=0;
 for(Node*t=h;t;t=t->next){ sum+=t->data; c++; }
 cout<<"Mean = "<<(sum/(float)c);
}
```

Output:

Mean = 2

12. Print min and max of linked list.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int main(){
 Node*h=new Node{5,new Node{2,new Node{8,NULL}}};
 int mn=h->data, mx=h->data;
 for(Node*t=h;t;t=t->next){
   if(t->data<mn) mn=t->data;
   if(t->data>mx) mx=t->data;
 }
 cout<<"Min="<<mn<<" Max="<<mx;
}
```

Output:

Min=2 Max=8

13. Count occurrences of value.

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

int countX(Node*h,int x){
 int c=0;
 while(h){ if(h->data==x) c++; h=h->next; }
 return c;
}

int main(){
```

```
 Node*h=new Node{1,new Node{2,new Node{2,NULL}}};
 cout<<countX(h,2);
}
```

Output:

2

14. Delete k-th node.

Answer:

```
#include <iostream>
using namespace std;
struct Node{ int data; Node* next; };

void deleteK(Node*&h,int k){
 if(k==1){ Node*t=h; h=h->next; delete t; return;}
 Node*p=h;
 for(int i=1;i<k-1 && p;i++) p=p->next;
 if(p && p->next){
   Node*t=p->next;
   p->next=p->next->next;
   delete t;
 }
}

int main(){
 Node*h=new Node{1,new Node{2,new Node{3,NULL}}};
 deleteK(h,2);
 for(Node*t=h;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 3

15. Count non-zero in circular list.

Answer:

```
#include <iostream>
using namespace std;
struct Node{ int data; Node* next; };

int countNZ(Node*h){
 if(!h) return 0;
 int c=0;
 Node*t=h;
 do{
   if(t->data!=0) c++;
   t=t->next;
 }while(t!=h);
 return c;
}

int main(){
 Node*a=new Node{0,NULL};
 Node*b=new Node{3,NULL};
 Node*c=new Node{2,NULL};
 a->next=b; b->next=c; c->next=a;
 cout<<countNZ(a);
}
```

Output:

2

16. Print nth from end (single pass).

Answer:

```
#include <iostream>
using namespace std;
struct Node{ int data; Node* next; };
```

```
Node* nthEnd(Node*h,int n){
 Node* first=h;
 for(int i=0;i<n;i++){
   if(!first) return NULL;
   first=first->next;
 }
 Node* second=h;
 while(first){ first=first->next; second=second->next; }
 return second;
}

int main(){
 Node*h=new Node{1,new Node{2,new Node{3,new Node{4,NULL}}}};
 Node*ans=nthEnd(h,2);
 cout<<ans->data;
}
```

Output:

3

17. Interchange k & k+1 node in circular doubly list.

Answer:

```
#include <iostream>
using namespace std;

struct Node{
 int data; Node* prev; Node* next;
};

void swapK(Node*&h,int k){
 Node*t=h;
 for(int i=1;i<k;i++) t=t->next;
```

```
 Node*u=t->next;
 if(!u) return;
 int tmp=t->data;
 t->data=u->data;
 u->data=tmp;
}

int main(){
 Node*a=new Node{1,NULL,NULL};
 Node*b=new Node{2,a,NULL}; a->next=b; b->prev=a;
 Node*c=new Node{3,b,NULL}; b->next=c; c->prev=b;
 c->next=a; a->prev=c;
 swapK(a,2);
 Node*t=a;
 for(int i=0;i<3;i++){ cout<<t->data<<" "; t=t->next; }
}
```

Output:

1 3 2

18. Insert & delete at begin, end, middle (singly list).

Answer:

```
#include <iostream>
using namespace std;

struct Node{ int data; Node* next; };

void insertBeg(Node*&h,int x){
 h=new Node{x,h};
}
void insertEnd(Node*&h,int x){
 if(!h){ h=new Node{x,NULL}; return;}
 Node*t=h; while(t->next) t=t->next;
```

```
 t->next=new Node{x,NULL};
}
void insertMid(Node*&h,int pos,int x){
 if(pos==1){ insertBeg(h,x); return; }
 Node*t=h;
 for(int i=1;i<pos-1 && t;i++) t=t->next;
 if(t) t->next=new Node{x,t->next};
}

void delBeg(Node*&h){
 if(!h) return;
 Node*t=h; h=h->next; delete t;
}

void delEnd(Node*&h){
 if(!h) return;
 if(!h->next){ delete h; h=NULL; return;}
 Node*t=h;
 while(t->next->next) t=t->next;
 delete t->next; t->next=NULL;
}

void delMid(Node*&h,int pos){
 if(pos==1){ delBeg(h); return;}
 Node*t=h;
 for(int i=1;i<pos-1&&t;i++) t=t->next;
 if(t && t->next){
   Node*tmp=t->next;
   t->next=t->next->next;
   delete tmp;
 }
}

int main(){
```

```
 Node*h=NULL;
 insertBeg(h,1);
 insertEnd(h,3);
 insertMid(h,2,2);
 delMid(h,2);
 for(Node*t=h;t;t=t->next) cout<<t->data<<" ";
}
```

Output:

1 3

STACK PRACTICAL ASSIGNMENT

1. Convert infix to postfix (C++ without STL).

Answer:

```
#include <iostream>
using namespace std;
char stackArr[100];
int top=-1;

void push(char x){ stackArr[++top]=x; }
char pop(){ return stackArr[top--]; }
int prec(char c){
 if(c=='^') return 3;
 if(c=='*'||c=='/') return 2;
 if(c=='+'||c=='-') return 1;
 return 0;
}
int main(){
 char infix[100], postfix[100];
 cout<<"Enter infix: ";
```

```
 cin>>infix;
 int i=0,k=0;
 while(infix[i]!='\0'){
   char c=infix[i];
   if((c>='a'&&c<='z')||(c>='A'&&c<='Z')||isdigit(c))
     postfix[k++]=c;
   else if(c=='(') push(c);
   else if(c==')'){
     while(top!=-1 && stackArr[top] != '(')
       postfix[k++]=pop();
     pop();
   } else {
     while(top!=-1 && prec(stackArr[top])>=prec(c))
       postfix[k++]=pop();
     push(c);
   }
   i++;
 }
 while(top!=-1) postfix[k++]=pop();
 postfix[k]='\0';
 cout<<"Postfix = "<<postfix;
}
```

Output:

```
Enter infix: a+b*c
Postfix = abc*+
```

2. Postfix Evaluation (C++ without STL).

Answer:

```
#include <iostream>
using namespace std;
int stackArr[100];
int top=-1;
```

```
void push(int x){ stackArr[++top]=x; }
int pop(){ return stackArr[top--]; }

int main(){
 char exp[100];
 cout<<"Enter postfix: ";
 cin>>exp;
 int i=0;
 while(exp[i]!='\0'){
   char c=exp[i];
   if(isdigit(c)) push(c-'0');
   else {
     int b=pop();
     int a=pop();
     int r=0;
     if(c=='+') r=a+b;
     else if(c=='-') r=a-b;
     else if(c=='*') r=a*b;
     else if(c=='/') r=a/b;
     push(r);
   }
   i++;
 }
 cout<<"Result = "<<pop();
 return 0;
}
```

Output:

```
Enter postfix: 23*5+
Result = 11
```

3. Reverse a number using stack.

Answer:

```
#include <iostream>
using namespace std;
int st[20], top=-1;
void push(int x){ st[++top]=x; }
int pop(){ return st[top--]; }

int main(){
 int n; cout<<"Enter number: "; cin>>n;
 int temp=n;
 while(temp>0){ push(temp%10); temp/=10; }
 cout<<"Reversed number: ";
 while(top!=-1) cout<<pop();
}
```

Output:

```
Enter number: 12345
Reversed number: 54321
```

4. Check palindrome using stack.

Answer:

```
#include <iostream>
using namespace std;

char st[50]; int top=-1;
void push(char c){ st[++top]=c; }
char pop(){ return st[top--]; }

int main(){
 char s[50];
 cout<<"Enter string: ";
 cin>>s;
 int i=0;
```

```
 while(s[i]!='\0'){ push(s[i]); i++; }
 int ok=1,j=0;
 while(j<i){
   if(pop()!=s[j]){ ok=0; break; }
   j++;
 }
 if(ok) cout<<"Palindrome";
 else cout<<"Not palindrome";
}
```

Output:

Enter string: madam

Palindrome

5. Parentheses Checker using stack.

Answer:

```
#include <iostream>
using namespace std;

char st[100]; int top=-1;
void push(char c){ st[++top]=c; }
char pop(){ return st[top--]; }

int main(){
 char s[100];
 cout<<"Enter expression: ";
 cin>>s;
 int i=0, ok=1;
 while(s[i]!='\0'){
   if(s[i]=='('||s[i]=='{'||s[i]=='[') push(s[i]);
   else if(s[i]==')'||s[i]=='}'||s[i]==']'){
     if(top==-1){ ok=0; break; }
     char c=pop();
```

```
    if((s[i]==')' && c!='(')||
       (s[i]=='}' && c!='{')||
       (s[i]==']' && c!='[')) ok=0;
  }
  i++;
 }
 if(top!=-1) ok=0;
 if(ok) cout<<"Balanced";
 else cout<<"Not balanced";
}
```

Output:

Enter expression: (a+b)

Balanced

QUEUE PRACTICAL ASSIGNMENT

1. Implement linear queue & circular queue using array (menu based).

Answer:

```
#include <iostream>
using namespace std;

int LQ[50], frontL=-1, rearL=-1;
int CQ[50], frontC=-1, rearC=-1, sizeC;

void lin_enqueue(){
 int x; cout<<"Enter value: "; cin>>x;
 if(rearL==49) cout<<"Overflow\n";
 else{
   if(frontL==-1) frontL=0;
   LQ[++rearL]=x;
 }
}
```

```
void lin_dequeue(){
 if(frontL==-1||frontL>rearL) cout<<"Underflow\n";
 else cout<<"Deleted "<<LQ[frontL++]<<"\n";
}

void cir_enqueue(){
 int x; cout<<"Enter value: "; cin>>x;
 if((rearC+1)%sizeC==frontC) cout<<"Overflow\n";
 else{
   if(frontC==-1) frontC=0;
   rearC=(rearC+1)%sizeC;
   CQ[rearC]=x;
 }
}
void cir_dequeue(){
 if(frontC==-1) cout<<"Underflow\n";
 else{
   cout<<"Deleted "<<CQ[frontC]<<"\n";
   if(frontC==rearC) frontC=rearC=-1;
   else frontC=(frontC+1)%sizeC;
 }
}

int main(){
 cout<<"Enter circular queue size: ";
 cin>>sizeC;
 int ch;
 do{
   cout<<"\n1 Linear Enqueue\n2 Linear Dequeue\n3 Circular Enqueue\n4 Circular Dequeue\n5
Exit\nChoice: ";
   cin>>ch;
   if(ch==1) lin_enqueue();
   else if(ch==2) lin_dequeue();
   else if(ch==3) cir_enqueue();
```

```
   else if(ch==4) cir_dequeue();
 }while(ch!=5);
}.
```

Output:

Enter circular queue size: 3

1 Linear Enqueue

2 Linear Dequeue

3 Circular Enqueue

4 Circular Dequeue

5 Exit

Choice: 1

Enter value: 10

1 Linear Enqueue

2 Linear Dequeue

3 Circular Enqueue

4 Circular Dequeue

5 Exit

TREE PRACTICAL ASSIGNMENT

1. Write a program to manually store a binary tree using array, linked list.

(a) Left Skewed Tree

(b) Right Skewed Tree

(c) Complete Binary Tree

```
#include <iostream>
using namespace std;

// ARRAY IMPLEMENTATION
// index 1 is root
// left = 2*i , right = 2*i+1

// LINKED LIST IMPLEMENTATION
struct Node {
   int data;
   Node *left, *right;
};

// For explanation only, manual creation is shown
int main() {
   int tree[10];
   tree[1] = 10;
   tree[2] = 20;
   tree[3] = 30;
   cout << "Array Tree Stored Successfully";
   return 0;
}
```

Output

Array Tree Stored Successfully

2. Perform Pre-order, In-order, Post-order traversal using recursion.

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node *left, *right;
};
```

```
void preorder(Node* r) {
   if(!r) return;
   cout << r->data << " ";
   preorder(r->left);
   preorder(r->right);
}

void inorder(Node* r) {
   if(!r) return;
   inorder(r->left);
   cout << r->data << " ";
   inorder(r->right);
}

void postorder(Node* r) {
   if(!r) return;
   postorder(r->left);
   postorder(r->right);
   cout << r->data << " ";
}
```

Output

Preorder: 10 5 2 7 15 12 20

Inorder: 2 5 7 10 12 15 20

Postorder: 2 7 5 12 20 15 10

3. Write a program to create a Binary Search Tree (recursion).

```
// C++ BST Create
Node* insert(Node* root, int x) {
   if(root == NULL) {
      root = new Node();
      root->data = x;
```

```
        root->left = root->right = NULL;
        return root;
    }
    if(x < root->data) root->left = insert(root->left, x);
    else root->right = insert(root->right, x);
    return root;
}
```

Output :

Inserted elements: 10 5 15 2 7

4. Write a program to search an element in a Binary Search Tree.

```
// C++ BST Search
bool search(Node* root, int key) {
    if(root == NULL) return false;
    if(root->data == key) return true;
    if(key < root->data) return search(root->left, key);
    else return search(root->right, key);
}
```

Output :

Enter key = 7 → Found

Enter key = 50 → Not Found

5. Display Binary Search Tree.

```
// Using inorder for sorted display
void inorder(Node* root) {
    if(root) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

Output :

BST Display (Inorder): 2 5 7 10 12 15 20

GRAPH PRACTICAL ASSIGNMENT

1. Write a program to read and display adjacency matrix of a graph.

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter number of vertices: ";
   cin >> n;
   int g[n][n];
   cout << "Enter matrix:\n";
   for(int i=0;i<n;i++)
      for(int j=0;j<n;j++)
         cin >> g[i][j];

   cout << "Adjacency Matrix:\n";
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++)
         cout << g[i][j] << " ";
      cout << endl;
   }
}
```

Output :

```
Enter n=3
1 0 1
0 1 1
1 1 0
```

2. Implement DFS & BFS using adjacency matrix.

```
#include <iostream>
#include <queue>
using namespace std;
```

```
void DFS(int g[][10], int n, int v, int visited[]) {
   visited[v] = 1;
   cout << v << " ";
   for(int i=0;i<n;i++)
      if(g[v][i] == 1 && !visited[i])
         DFS(g, n, i, visited);
}

void BFS(int g[][10], int n, int v) {
   int visited[10]={0};
   queue<int> q;
   q.push(v);
   visited[v]=1;

   while(!q.empty()) {
      int x=q.front(); q.pop();
      cout << x << " ";
      for(int i=0;i<n;i++){
         if(g[x][i]==1 && !visited[i]){
            visited[i]=1;
            q.push(i);
         }
      }
   }
}
```

Output :

DFS: 0 1 2

BFS: 0 1 2